

Typescript

Carta di identità

Typescript aumenta la sintassi di Javascript con un **sistema di tipi**

In Typescript possiamo dichiarare il tipo atteso di variabili, parametri e valori di ritorno delle funzioni.

Typescript si **compila**, i programmi TS vengono tradotti in programmi JS

Dunque i costrutti «aggiuntivi» di TS sono **esclusivamente dichiarativi e non operazionali**: «scompaiono» effettuata la compilazione e non hanno nessun impatto sulla **esecuzione** del codice.

Typescript implementa un **sistema di inferenza dei tipi**

In fase di compilazione, TS determina il tipo risultante di funzioni ed espressioni, e verifica la consistenza con i tipi «promessi» come valori di ritorno, o con i modi in cui i risultati delle espressioni sono utilizzati.

Typescript è al suo meglio quando **integrato** in un **IDE**

L'inferenza sui tipi viene applicata già mentre scriviamo il codice, senza aspettare la compilazione. In questo modo è possibile prevenire molti errori di programmazione.

La differenza fra tipi statici e tipi dinamici

In Typescript possiamo dichiarare nel codice il tipo di variabili, parametri e valori di ritorno delle funzioni.

«Possiamo» e non «dobbiamo» perché in Typescript è possibile **non** esprimere il tipo di una di queste entità semplicemente scrivendo che il tuo tipo è **any**, «qualsiasi cosa». Tuttavia fare ciò fa sì che buona parte dei vantaggi di Typescript si perdano, quindi a parte casi molto rari in cui è necessario far andare d'accordo del codice scritto in Typescript con del codice scritto in Javascript e dunque privo di dichiarazioni di tipo, è meglio evitare l'uso di **any**.

Le dichiarazioni di tipo in Typescript non hanno nessun effetto sulla **esecuzione** del codice. Mentre in linguaggi come Javascript o C il tipo delle variabili e dei parametri influenza lo slot di memoria (statica o dinamica) utilizzato per memorizzarvi il dato, in Typescript questo non succede. Typescript, infatti, viene **compilato** in Javascript, e una volta effettuata questa operazione le dichiarazioni di tipo scompaiono.

A cosa servono allora le dichiarazioni di tipo in Typescript?

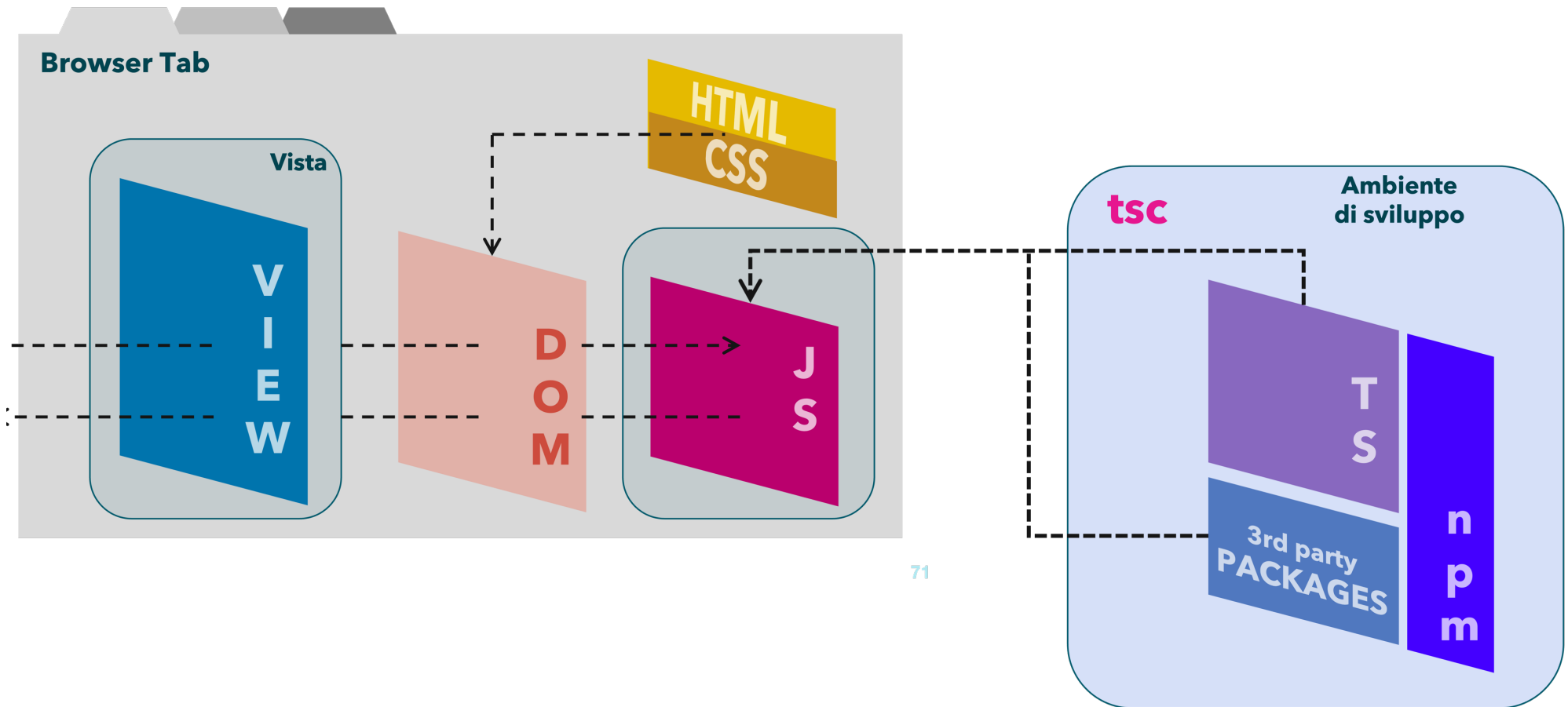
A far sì che **a compile-time** sia possibile verificare la consistenza dei tipi nelle espressioni e negli assegnamenti. Typescript è in grado di inferire il tipo di un'espressione a partire dai tipi degli elementi che la costituiscono, e dunque verificare che la variabile (o parametro) in cui il risultato dell'espressione viene memorizzato corrisponda.

Questo previene molti errori in fase di esecuzione. Dichiarare i tipi permette ai programmatori di esprimere le proprie intenzioni, e permette al compilatore (nonché all'ambiente di sviluppo, che spesso sbriga parte del lavoro in anticipo) di rilevare possibili discrepanze fra «l'intenzione» del programmatore e il codice che ha effettivamente scritto.

Permette inoltre di controllare che le proprietà e i metodi di un oggetto a cui si accede nel codice esistano effettivamente (grazie al fatto che il tipo dell'oggetto è stato dichiarato) e permette altresì di introdurre nelle classi i modificatori di accesso (public, private, ecc) che in Javascript erano assenti, introducendo un ulteriore livello di controllo per prevenire eventuali bug.

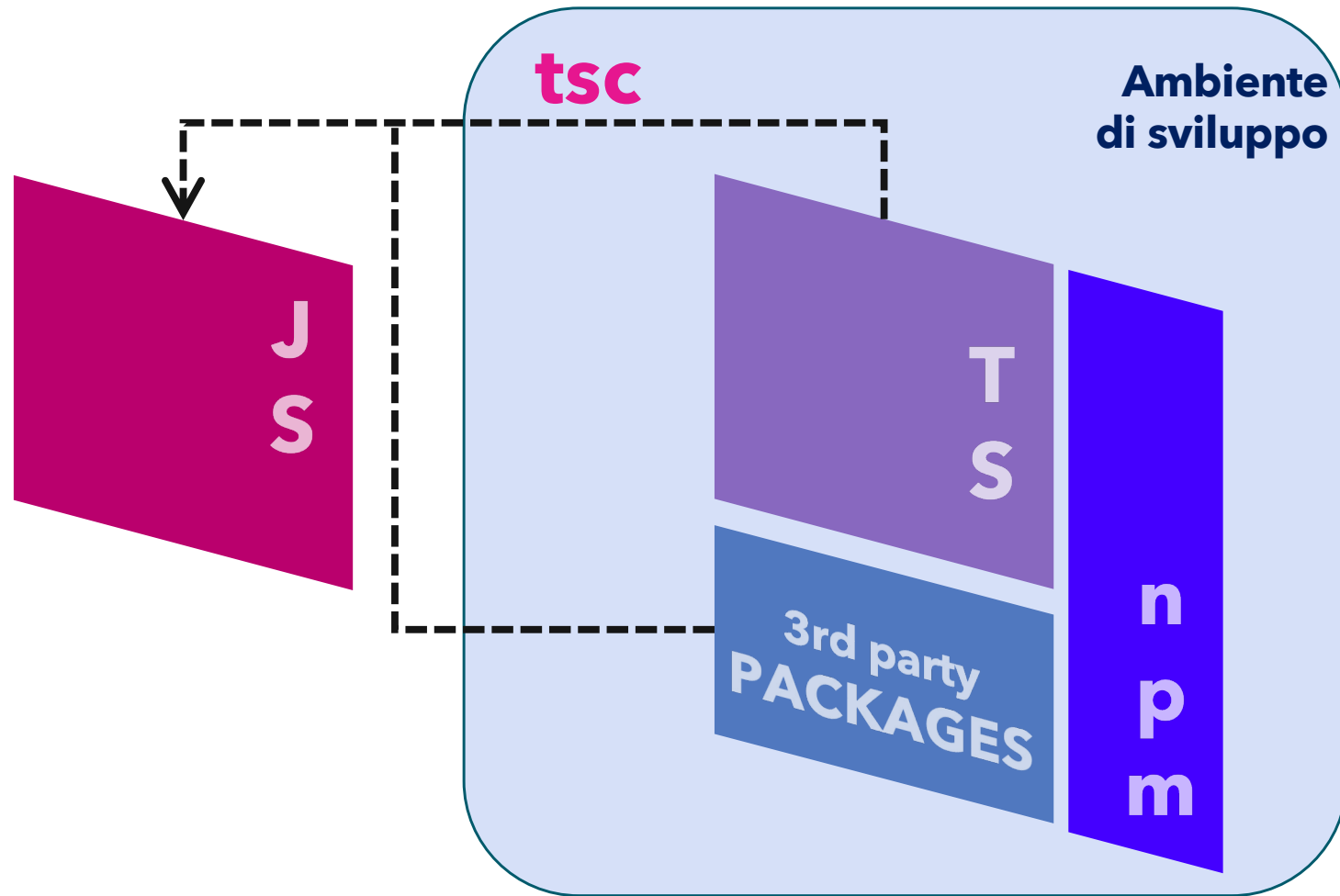
Typescript

Uno strumento per l'ambiente di sviluppo



Typescript

Uno strumento per l'ambiente di sviluppo



Typescript, l'ambiente di sviluppo e la gestione delle dipendenze

L'uso di Typescript ci permette di introdurre una questione fondamentale relativa all'ambiente di sviluppo e alla gestione di progetto di una moderna applicazione web: la **gestione delle dipendenze**

Con l'introduzione nei nostri progetti applicativi di Typescript, infatti, essi iniziano a **dipendere** dal compilatore Typescript (che si chiama **tsc**), e dalle librerie, o, come è più comune dire nel contesto delle applicazioni web, dai **package**, che definiscono il linguaggio stesso e che il compilatore utilizza.

Si tratta di un'istanza particolare di un caso più generale: Typescript o no, nello sviluppo web è estremamente comune utilizzare **package di terze parti**, al punto che si è creata una fitta rete di sviluppatori che condividono gratuitamente moduli e funzioni da loro progettate. Questi package sono distribuiti tramite **repository online** che forniscono anche un minimo di statistiche sull'uso, sul gradimento e sull'aggiornamento dei vari package, in modo da poter decidere con cognizione di causa se e quali utilizzare.

Anche il package più affidabile tuttavia prima o poi verrà aggiornato (anzi, forse conviene diffidare un po' di package che non sono aggiornati da anni, a meno che non svolgano compiti molto semplici e consolidati). E se quel package usa altri package... la catena degli aggiornamenti potrebbe complicarsi un bel po'.

Quand'è che è necessario che io riconfezioni e ridistribuisca il mio progetto perché lo richiedono i package da cui esso dipende? Quando devo modificarne il codice perché qualche package che ho usato ha cambiato la signature di qualche funzione? Come evitare che il legittimo aggiornamento di un package «rompa» applicazioni già distribuite che si erano basate su di esso? Cosa succede se uno sviluppatore «ritira» il proprio package e smette di distribuirlo?

Package Management

Una risposta importante a questi interrogativi è data dall'utilizzo ormai pervasivo di strumenti di **package management**, in abbinamento a **package registries** online che offrono la possibilità di pubblicare e condividere package fra sviluppatori.

Un package manager è un tool, tipicamente da linea di comando, che permette di accedere al registry/repository ad esso abbinato, scaricare ed installare i package che servono all'applicazione, e dichiarare la dipendenza da essi tramite opportuni **dependency files**.

Il package manager che utilizzeremo è quello più diffuso in assoluto nel mondo Javascript/Typescript: **npm (node package manager)**.

Inizialmente **npm** nasce come tool di package management per l'ambiente **Node.js**. Tuttavia è ormai da tempo un progetto separato, attualmente «costola» di GitHub. Il repository/registry associato a **npm** è **npmjs.com**. Il modo più semplice di installare **npm** resta comunque installare Node.js, che lo include.

npm svolge anche altri compiti oltre al package management ma le vedremo quando e se ci serviranno... una trattazione estremamente approfondita di **npm** esula dagli obiettivi di questo corso.

Project dependencies

Dipendenze locali vs dipendenze globali

npm permette di installare i package sia a livello **globale** (come librerie del s.o.) sia a livello **locale**, nella cartella del progetto.

per le modalità contemporanee di sviluppo software, la strategia più comune è di installare i package a livello locale: questo rende lo sviluppo indipendente dalla configurazione locale del dispositivo utilizzato per lo sviluppo, permettendo di collaborare al meglio fra persone diverse e dispositivi diversi, tramite strumenti come **Git** o **GitLab**.

Dipendenze dev vs dipendenze runtime

npm (e i tool di dependency management in generale) distingue due tipi di dipendenze da un package:

- **Dev dependencies:** sono i package da cui il progetto dipende quando è in fase di sviluppo, ma che «scompariranno» una volta che il progetto sarà distribuito ed eseguito. Typescript è una di queste: quando il progetto sarà compilato in Javascript, le librerie e il compilatore typescript non serviranno più.
- **Runtime dependencies:** sono i package da cui il progetto dipende in fase di esecuzione: contengono codice che viene effettivamente invocato durante l'esecuzione del progetto, dunque sarà importante che questo codice sia incluso nella distribuzione del progetto stesso.

Questa distinzione è rilevante quando si costruiscono progetti complessi, che, dopo lo sviluppo, attraversano una fase di «confezionamento» al fine di essere distribuiti. Il «confezionamento» viene solitamente svolto attraverso tool che necessitano di sapere cosa va incluso nella confezione e cosa va lasciato fuori. L'obiettivo è che da una parte il progetto e il codice in corso di sviluppo siano il più leggibili possibile, dall'altra che il progetto da distribuire sia il più conciso ed efficiente possibile.

Inizializzazione e uso di base di npm

Command Line Interface di base

Il modo più semplice di installare **npm** è installare **Node.js**.

<https://nodejs.org/en/download>

A quel punto la **CLI** di **npm** dovrebbe risultare disponibile da terminale (anche il terminale interno di WebStorm o VisualStudio Code)

npm inizia a gestire un progetto quando lo usiamo per **installare** il primo package nel progetto stesso

```
> npm install [-D] [-g] nome-package[@versione]
```

-D indica che si tratta di una **dev dependency**.
Il default è **runtime dependency**.

-g indica di installare **globalmente**.
Il default è **installazione locale** al progetto.

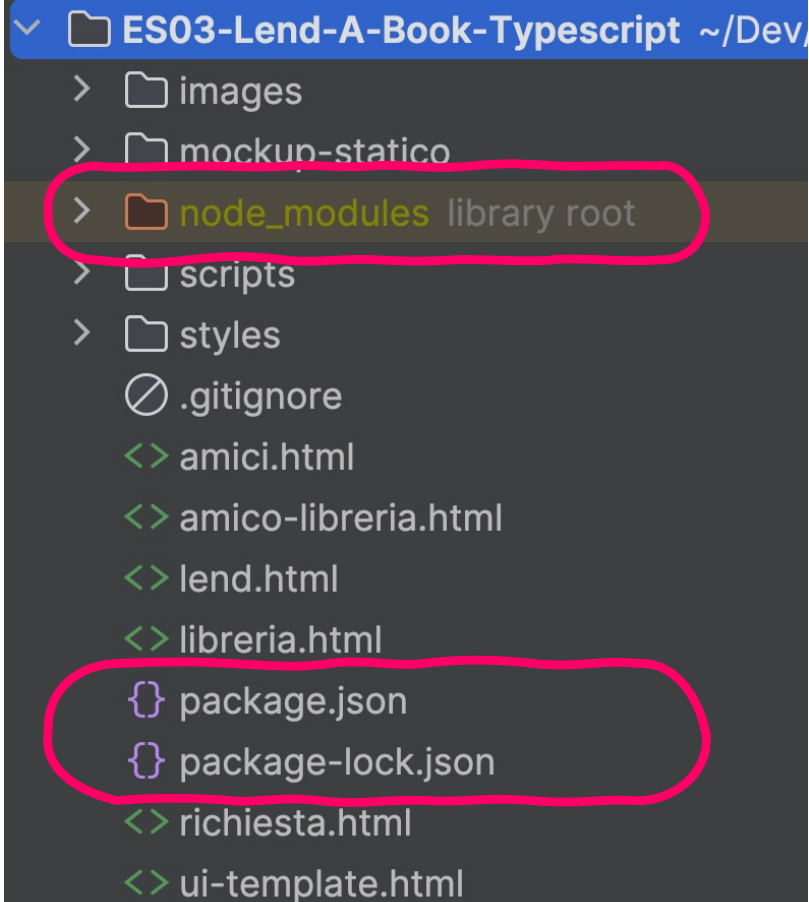
@versione specifica quale versione del package va installata, tramite il **numero di versione**.
@latest indica l'ultima stabile.
@next indica la versione di prossima uscita (tipicamente non stabile e ancora soggetta a modifiche)

```
> npm install -D typescript@latest
```


Inizializzazione e uso di base di npm

Struttura di progetto

STRUTTURA PROGETTO



ES03-Lend-A-Book-TypeScript ~/Dev

- > images
- > mockup-statico
- > **node_modules** library root
- > scripts
- > styles
- ⊗ .gitignore
- <> amici.html
- <> amico-libreria.html
- <> lend.html
- <> libreria.html
- package.json**
- package-lock.json**
- <> richiesta.html
- <> ui-template.html

node_modules: il codice dei package; questa cartella è interamente gestita da **npm** e non va modificata. **Se usiamo git per condividere il progetto, va *esclusa*!!!**

package.json: la configurazione del progetto e le dipendenze; **npm** scrive in questo file ma possiamo modificarlo anche noi a mano

```
{  
  "devDependencies": {  
    "typescript": "^5.6.3"  
  }  
}
```

package-lock.json: dipendenze e sotto-dipendenze, file generato da **npm** da non modificare.

Inizializzazione e configurazione di Typescript

Il compilatore typescript (**tsc**) è **uno script javascript** contenuto nel relativo **package**, ed eseguito tramite il tool **npx**, incluso in npm.

Per prima cosa dobbiamo inizializzare il progetto all'uso in **typescript**.

```
> npx tsc --init
```

tsc non compila ancora nulla, ma crea un file di configurazione **tsconfig.json** inserendo alcune impostazioni predefinite, che, per la nostra tipologia di progetto, dovremo parzialmente modificare.

```
{
  "compilerOptions": {
    "target": "es2016", /* --> es2020 */
    "module": "commonjs", /* --> es2015 */
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true
  }
}
```

Organizzazione di un progetto Typescript

STRUTTURA PROGETTO

ES03-Lend-A-Book-Typecript ~/Dev/TWeb/

- > images
- > mockup-statico
- > node_modules library root
- > scripts
- > src
- > styles
- ⊗ .gitignore
- <> amici.html
- <> amico-libreria.html
- <> lend.html
- <> libreria.html
- { } package.json
- { } package-lock.json
- <> richiesta.html
- { } tsconfig.json
- <> ui-template.html

```
{  
  "compilerOptions": {  
    "target": "es2016",  
    "module": "commonjs", /* --> es2015 */  
    "esModuleInterop": true,  
    "forceConsistentCasingInFileNames": true,  
    "strict": true,  
    "skipLibCheck": true,  
    "outDir": "scripts" <<<<  
  }  
}
```

Configurazione di Typescript (tsconfig.json) – Parte I

Opzioni del compilatore (**compilerOptions**)

- **target:** permette di specificare la versione di Javascript in cui deve essere scritto l'output della compilazione. Il default è **es2016** in quanto è una versione di Javascript che può essere eseguita da tutti i browser in circolazione. Noi useremo **es2020** per avere accesso a un numero maggiore di feature del linguaggio. In ogni caso anche es2020 è ormai ampiamente supportata.
- **module:** prima che la specifica ufficiale EcmaScript introducesse l'**import/export** nel linguaggio, l'ambiente di esecuzione Node.js aveva proposto una propria sintassi per gestire i moduli, nota con il nome «Common JS». Nel tempo Node.js si è evoluto e ora supporta l'interoperabilità fra i due diversi modi di specificare i moduli (in questo caso, useremmo l'opzione **nodenext**). Tuttavia poiché stiamo scrivendo per il browser, e siamo interessati a mantenere la sintassi di EcmaScript, indichiamo **es2015** (è la versione di ES in cui sono stati introdotti import/export).
- **esModuleInterop:** serve a garantire l'interoperabilità fra le due modalità di specifica di moduli, quella di Node e quella di ES. L'interoperabilità serve principalmente se stiamo programmando per l'ambiente Node ma vogliamo poter dichiarare i moduli con lo stile di ES. In caso contrario il valore di questa proprietà è indifferente.
- **forceConsistentCasingInFileNames:** fa sì che il compilatore si accorga se uno stesso modulo viene importato con un uso inconsistente di maiuscole/minuscole (il che potrebbe succedere se uno sviluppatore ha un sistema operativo che non opera la distinzione, ma causerebbe problemi ad eventuali suoi collaboratori con sistemi operativi diversi)
- **strict:** se **true**, applica maggiore severità nella verifica della consistenza di tipo
- **skipLibCheck:** se **true**, non verifica le consistenze dei tipi all'interno delle librerie/package

Per una trattazione estensiva di **tutte** le opzioni disponibili per la configurazione di un progetto Typescript, è possibile consultare la documentazione di **tsconfig**:

<https://www.typescriptlang.org/tsconfig/>

Configurazione di Typescript (tsconfig.json) – Parte II

Opzioni del compilatore (**compilerOptions**)

Converrà avere una cartella separata in cui tenere i sorgenti **typescript**, indicando al compilatore dove deve salvare i file **javascript** risultanti dalla compilazione.

Questi ultimi costituiscono la «vera» applicazione che verrà distribuita; dunque le pagine html si riferiscono ad essa e ignorano l'esistenza della cartella con i sorgenti. I sorgenti, in fase di distribuzione dell'applicazione e/o di sua esecuzione non servono più.

Quindi per il nostro progetto possiamo operare così:

- I file javascript continueranno a stare nella cartella **scripts**
- Per i file typescript introduciamo una nuova cartella **src**.

Non è necessario dire al compilatore dove trovare i sorgenti typescript (li trova da sé). Dobbiamo invece indicare, sempre tramite una **compilerOption**, dove mettere i file risultanti:

- **outDir**: "nome-cartella-output"

Il compilatore replicherà la struttura di file e sottocartelle dei sorgenti typescript all'interno della cartella **scripts**, sostituendo però i file typescript con le loro controparti compilate.

Come già detto, le pagine html importano gli script JS che si trovano in **scripts**: se vorremo pubblicare l'applicazione su un web server, è questa cartella che dovremo includere. Viceversa, se dobbiamo collaborare con altri sviluppatori (es. via GitLab), a loro serve solo la cartella **src**, in quanto la cartella scripts viene generata automaticamente all'atto della compilazione.

Compilare i sorgenti Typescript

Il compilatore typescript (**tsc**) è **uno script javascript** contenuto nel relativo **package**, ed eseguito tramite il tool **npx**, incluso in npm.

Il compilatore ricerca automaticamente i file **.ts** a partire dalla cartella in cui lo eseguiamo, e memorizza i risultanti file **.js** nella **outDir** che abbiamo indicato in **tsconfig.json**

```
> npx tsc
```



```
✓ scripts
  JS test.js
✓ src
  TS test.ts
```

In generale, la struttura di cartelle e sottocartelle in cui organizziamo i nostri sorgenti **typescript** verrà automaticamente replicata nella cartella indicata per l'output (nel nostro caso, **scripts**), solo con i file **.js** al posto dei file **.ts**.

Dichiarazioni di tipo in Typescript

Variabili, parametri e tipi di ritorno

Variabili

```
let nomevar: TIPO;  
const c: TIPO = espressione;
```

Parametri

```
function fun(p: TIPO, q: TIPO) {...}  
const arr = (a: TIPO, b: TIPO) => ...
```

Valori di ritorno

```
function fun(...): TIPO {...}  
const arr = (...): TIPO => ...
```

Le dichiarazioni di tipo di **variabili** e **parametri** sono **obbligatorie**, a meno che non venga immediatamente assegnato un valore, nel qual caso Typescript **inferirà** il tipo in base all'espressione del valore assegnato.

La dichiarazione di tipo del **valore di ritorno** di una funzione non è obbligatoria, in quanto Typescript lo inferisce dai tipi dei valori restituiti nelle **return**.

Tranne in casi banali, è tuttavia utile **dichiarare esplicitamente** i tipi anche in questi casi: in questo modo esprimiamo le nostre intenzioni, e se nel codice c'è un errore per cui, ad esempio, il valore che restituiamo non è del tipo corretto, o magari ci dimentichiamo di mettere la return, il compilatore potrà segnalarcelo.

Espressioni di tipo in Typescript – tipi base e tipi array

Imparare Typescript consiste principalmente nel saper scrivere le espressioni di tipo. Questo è talvolta meno banale di quanto sembri. Typescript è un'estensione di Javascript, dunque una variabile potrebbe avere come tipo «una funzione che prende in input un array di interi e restituisce una stringa» o «un oggetto usato come array associativo in cui gli indici sono stringhe e i valori sono numeri oppure booleani».

Scrivere correttamente le espressioni di tipo tuttavia è in sostanza una forma preliminare di testing della nostra applicazione! Se il nostro codice supera i controlli di tipo, abbiamo già escluso una serie di errori grossolani ma molto frequenti.

Le espressioni di tipo più semplici in Typescript sono le seguenti:

- Il **nome** di un tipo base: **string**, **boolean**, **number**, **undefined**, **null**, **bigint**, **symbol** (anche se gli ultimi due non li abbiamo approfonditi)
- Il **nome** di una classe, a indicare che la variabile conterrà solo oggetti di quella classe
- Il tipo **void**, che si usa per esprimere il tipo di valore di ritorno di una funzione che non ritorna nulla.
- Il **tipo array**, che si ottiene facendo seguire al tipo degli *elementi contenuti nell'array* le parentesi quadre:

```
let mioArray: number[];
```

```
let iscrittiAllEsame: Studente[]; /* dove Studente è una classe */
```

- Il **tipo tupla**, che esprime un array con un numero finito e predeterminato di elementi, di cui si conoscono i tipi, che possono essere diversi fra loro; si esprime elencando i tipi degli elementi fra parentesi quadre:

```
let tripla: [number, string, Studente]
```


Espressioni di tipo in Typescript – Tipo unione

A partire dalle espressioni di tipo più semplici, è possibile esprimere tipi complessi.

- **Tipo unione.** È il tipo risultante dall'unione di altri due tipi. Corrisponde all'idea che una variabile (o parametro, o valore di ritorno) possa essere di un tipo **A** oppure di un tipo **B** e si esprime componendo i due tipi in questione con l'operatore `|` (barra verticale):

`tipoA | tipoB`

Ad esempio, un array potrebbe contenere interi **oppure** stringhe:

```
let misto: (number | string)[]
```

oppure (cosa assai frequente) una funzione potrebbe restituire un oggetto della classe `Studente`, **oppure** `null`:

```
function trovaStudente(matricola: number): Studente | null {...}
```

Il tipo unione può essere usato anche per esprimere un tipo come **elenco di possibili valori**, ad esempio:

```
let cifraDecimale: 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9;  
let primavera: "marzo" | "aprile" | "maggio";
```

Espressioni di tipo in Typescript – Tipo funzione

A partire dalle espressioni di tipo più semplici, è possibile esprimere tipi più complessi.

- **Tipo funzione.** Qual è il tipo di una variabile, o di un parametro, atto a contenere una funzione? L'espressione di tipo si costruisce a partire dal tipo dei parametri e del valore di ritorno della funzione che vogliamo memorizzare nella variabile, in questo modo:

$$(p1: TIPO1, \dots, pN: TIPON) \Rightarrow TIPORET$$

Va notato che in un'espressione di tipo siffatta i «nomi» $p1, \dots, pN$ sono solo dei segnaposto, non sono i nomi dei parametri della funzione che memorizzeremo nella variabile.

Come esempio, consideriamo il caso di una funzione **superCalcolo** che lancia un calcolo complesso e computazionalmente oneroso su un computer remoto; questa funzione prende come argomento una funzione di callback che dovrà «ricevere» il risultato del calcolo, sotto forma di array di interi, ed elaborarlo, restituendo infine un valore true/false che esprima se il risultato è soddisfacente o necessita di ulteriori passaggi di calcolo. La funzione **superCalcolo** potrebbe avere questa signature:

```
function superCalcolo(datiInput: number[],  
    callback: (datiOutput: number[]) => boolean): void {...}
```

E se volessimo memorizzare l'intera funzione **superCalcolo** in una variabile, di che tipo dovrebbe essere?

```
let somevar: ?;  
somevar = superCalcolo;
```

Il tipo di **somevar** potrebbe essere espresso così;

```
let somevar: (x: number[], y: (z: number[]) => boolean) => void;
```

Espressioni di tipo in Typescript – Le classi

Abbiamo già visto che il nome di una classe è un'espressione di tipo: indica l'insieme degli oggetti che a quella classe appartengono.

La definizione di una classe in Typescript tuttavia caratterizza in modo preciso gli oggetti che a quella classe appartengono, in modo simile a quello che accade in altri linguaggi OOP. In Typescript dunque una classe dichiara **le proprietà** (o **variabili di istanza**) degli oggetti che le appartengono, con i relativi tipi:

```
class Studente {  
    nome: string;  
    matricola: number;  
    votiEsami: number[];  
    ...  
}
```

Per il resto, la definizione della classe specifica il costruttore e i metodi, in modo analogo a quanto accade in vanilla Javascript, a parte il fatto che dichiareremo i tipi dei parametri e dei valori di ritorno.

È possibile in Typescript altresì dichiarare la visibilità delle variabili d'istanza: le keyword sono quelle già note, **private**, **public** (il default) o **protected**. In più, è possibile usare la keyword **readonly**, a indicare una variabile d'istanza che viene assegnata nel costruttore e mai più modificata.

Typescript verifica che tutte le variabili di istanza vengano inizializzate nel costruttore; se vogliamo evitare questo controllo (ad esempio, perché sappiamo che una di esse verrà inizializzata in seguito in conseguenza di qualche evento successivo alla creazione dell'oggetto) possiamo posporre al nome della variabile in questione il simbolo ! (punto esclamativo). Possiamo inoltre dichiarare che una variabile d'istanza è opzionale (ossia, potrebbe non avere assegnato alcun valore) posponendo il simbolo ? (punto di domanda).

In Typescript, le proprietà e i metodi di un oggetto appartenente a una data classe devono **sempre** rispettare la dichiarazione della classe. Dunque le uniche proprietà che è possibile aggiungere o togliere dagli oggetti di una classe durante il corso della loro vita sono quelle opzionali, ossia quelle dichiarate con il simbolo ?.

Espressioni di tipo in Typescript – Interfacce

Le **interface**, come già detto in precedenza, sono un costrutto completamente dichiarativo; non hanno alcuna semantica operativa.

Infatti, non ne troviamo traccia nel codice compilato.

L'utilità delle **interface** consiste nel garantire, all'interno del codice, che gli oggetti ricevuti in input o elaborati come output abbiano determinate proprietà. Questo è particolarmente importante in un linguaggio dove è possibile (e frequente) creare oggetti senza passare per dei costruttori.

Vediamo un esempio di **interface**:

```
interface Book {  
  title: string;  
  author: string;  
  year: number  
}
```

Come si può notare, una interface è semplicemente un elenco di proprietà (volendo, anche proprietà-funzione).

Il nome di una interface (nell'esempio, Book) definisce un tipo. Un oggetto **rispetta** una interface (e dunque può dirsi del tipo corrispondente) se **possiede** le proprietà elencate.

Una interface può dichiarare proprietà **opzionali** (che sono ammesse, ma non obbligatorie, per dire di rispettare la interface) mettendo un ? (punto di domanda) subito dopo il nome della proprietà.

Per quanto riguarda il modo di determinare se un dato oggetto *ogg* **rispetta** una data interfaccia *I* dobbiamo distinguere due casi:

- un **object literal** rispetta *I* se le corrisponde esattamente in termini di proprietà; in altre parole deve avere tutte le proprietà non opzionali, e **non avere proprietà aggiuntive**
- un **oggetto appartenente a una classe C** rispetta *I* se *C* dichiara tutte le proprietà non opzionali di *I*; tuttavia *C* potrebbe dichiarare anche proprietà aggiuntive.

Una classe *C* che definisce tutte le proprietà di un'interfaccia *I* si dice che **implementa** l'interfaccia; questo può essere esplicitato attraverso il costrutto *implements*:

```
class C implements I {...}
```

Espressioni di tipo in Typescript – Interfacce (continua)

La versione «Typescript» della DOM API definisce formalmente come **interface** tutti i tipi di oggetti (nodi del DOM, oggetti evento, ecc) che utilizza.

Molte API, package e librerie originariamente scritti in Javascript forniscono parallelamente delle definizioni di tipo per gli oggetti, le classi e le funzioni che mettono a disposizione, in modo da poter essere utilizzati con Typescript. Queste definizioni sono tipicamente contenute in file con l'estensione **.d.ts**.

Ad esempio, i tipi di dati della DOM API sono dichiarati in un file **lib.dom.d.ts** contenuto nel package **typescript** che abbiamo scaricato e installato con **npm**.

Espressioni di tipo in Typescript – Dizionari indicizzati

In Javascript, e di conseguenza in Typescript, è piuttosto comune usare gli oggetti come «dizionari indicizzati» (in modo simile a quello in cui si userebbe una hash map) per memorizzare, e reperire facilmente, dei valori a partire da una chiave.

Supponiamo ad esempio di voler memorizzare le informazioni anagrafiche di un certo numero di persone (impiegati di un'azienda, iscritti a un club, abitanti di una circoscrizione...) e di volerle reperire facilmente a partire dal codice fiscale. Potremmo utilizzare un oggetto «repository» in cui le chiavi (ossia, i nomi delle proprietà) sono appunto i codici fiscali, e a ciascuna di esse corrisponde un oggetto della classe Persona, che contiene al suo interno l'anagrafica della persona in questione. La cosa potrebbe a grandi linee funzionare così:

```
const repo = {};  
function memorizza(p: Persona) {  
    repo[p.codiceFiscale] = p;  
}
```

Purtroppo però questo codice in Typescript non funziona. Typescript non permette (a differenza di Javascript) di accedere a proprietà di un oggetto che non siano esplicitamente menzionate dalla sua definizione di tipo. L'unico caso in cui lo permette è quando l'oggetto è esplicitamente dichiarato come **«dizionario indicizzato»** di coppie chiave/valore.

Il tipo di **repo** dovrà dunque essere dichiarato così:

```
const repo: {[chiave: string]: Persona}
```

Questa dicitura di tipo afferma che **repo** è un dizionario indicizzato in cui le chiavi sono stringhe e i valori sono di tipo Persona. Possiamo definire dizionari indicizzati con tutti i tipi di valori che vogliamo, ma le chiavi potranno essere soltanto stringhe o numeri.

Accessors (getters e setters) - I

L'aver introdotto le **interface** ci permette di introdurre un costrutto del linguaggio il cui significato potrebbe altrimenti risultare un po' oscuro: i **getter** e i **setter**. Si tratta di una caratteristica di Javascript, non di Typescript, ma la sua utilità è assai più evidente quando abbiamo a che fare con le dichiarazioni di tipo.

Rimanendo nell'esempio delle informazioni anagrafiche, immaginiamo un'interfaccia di questo tipo:

```
interface RagioneSociale {  
    denominazione: string;  
    codiceFiscale: string;  
}
```

Sia le persone fisiche che le aziende o gli enti potrebbero implementare questa interfaccia. Nel caso della classe *Persona*, le due proprietà «denominazione» e «codice fiscale» possono essere calcolate a partire da altre proprietà già presenti nella classe stessa, come il nome, il cognome, il luogo e la data di nascita. Sarebbe quindi più utile se fossero metodi, ma l'interfaccia ci richiede che siano proprietà.

La soluzione in questo caso sono i **getter** e i **setter**, tramite i quali è possibile definire proprietà «calcolate»:

```
class Persona {  
    get denominazione(): string {  
        return `${this.nome} ${this.cognome}`;  
    }  
    set denominazione(den: string): void {  
        const sep = den.split(" ");  
        this.nome = sep[0]; this.cognome = sep[1];  
    }  
}
```


Accessors (getters e setters) - II

La proprietà **denominazione** così definita rispetta l'interfaccia RagioneSociale.

Essa verrà utilizzata nel codice come una proprietà a tutti gli effetti, dato un oggetto **tizio** appartenente alla classe **Persona** sarà dunque possibile scrivere:

```
tizio.ragioneSociale = "Mickey Mouse";  
const someVar = tizio.ragioneSociale;
```

Per definire una proprietà tramite getter/setter:

- L'accessor **get** non deve prendere parametri, ma deve restituire il valore della proprietà
- Viceversa, l'accessor **set** non deve restituire nulla, ma deve prendere come parametro il nuovo valore della proprietà.
- Se sono presenti entrambi gli accessor per la stessa proprietà, il tipo di ritorno della **get** deve corrispondere al tipo del parametro della **set**
- Non è obbligatorio che ci siano entrambi: una proprietà con solo **get** sarà **readonly**; una proprietà con solo **set** sarà scrivibile ma non leggibile (anche se è più difficile che si presenti questo tipo di necessità).

Tutte le volte che la proprietà apparirà come **right-hand side** di un assegnamento, il valore dell'espressione a sinistra dell'uguale verrà passato al metodo setter. Tutte le volte che la proprietà apparirà in un'espressione **left-hand side**, ossia sarà necessario conoscere il suo valore, per ottenerlo verrà invocato il metodo getter.

I setter e i getter sono utilizzati, oltre che in situazioni come quella esemplificata, per esprimere proprietà che possono essere calcolate a partire da altri valori già memorizzati, o per controllare maggiormente l'accesso alle proprietà interne di un oggetto, senza essere costretti a renderle private. Si pensi ad esempio a proprietà numeriche di cui si deve limitare il valore o a proprietà testuali che richiedono un controllo sul formato o sul tipo di caratteri utilizzati; o ancora a valori interni che devono essere in qualche modo elaborati prima di essere restituiti verso l'esterno.

Generics

I *generics* (letteralmente, «generici») sono elementi del linguaggio che esprimono un comportamento che è indipendente dal tipo di dato su cui operano. Alcuni esempi:

- Un algoritmo di ordinamento come il bubble-sort può operare su array di tipi diversi; in ogni caso restituirà un array (ordinato) dello stesso tipo di quello ricevuto in input
- Una funzione che restituisce l'elemento «centrale» (come posizione) di un array, opera nello stesso modo indipendente dal tipo di array. Ricevuto un array di tipo **X[]** restituirà un singolo oggetto di tipo **X**.
- Una classe che rappresenta una struttura dati **Stack** (pila), con le operazioni **push** e **pop**, avrà un comportamento analogo a prescindere dal tipo di elementi che ci inseriamo; se abbiamo uno **Stack** di oggetti di tipo **X**, l'operazione **push** prenderà un dato di tipo **X** e lo metterà in cima alla pila; l'operazione **pop** rimuoverà l'elemento in cima alla pila, sempre di tipo **X**, e lo restituirà.

Ovviamente non vorremmo dover definire una nuova funzione, o una nuova classe, per ogni tipo su cui ci serve utilizzarla, se il comportamento è sempre identico.

Typescript permette dunque di definire **funzioni**, **classi**, o **interfacce** generici rispetto al tipo. Questa possibilità è utilizzata, ad esempio, in classi di libreria come **Set** o **Map**:

```
const insiemeNum = new Set<number>()  
const insiemeStudenti = new Set<Studente>()
```

Fra parentesi angolari viene espresso il tipo di oggetti su cui opera la funzione o la classe (nell'esempio, entrambe le cose: se una classe è generica, anche il suo costruttore lo è).

Lo stesso tipo di sintassi, usato però con una **variabile di tipo**, permette di creare elementi generici:

```
function middle<T>(lista: T[]): T {  
    return lista[lista.length/2];  
}
```

In questo esempio, **T** è la variabile di tipo.